

Polimorfisme dengan Metode Magic di Python

Rudiman, S.Kom.,M.Sc.

Polimorfisme dengan Metode Magic di Python

- Metode Magic atau **dunder methods** adalah metode khusus di Python yang diawali dan diakhiri dengan dua garis bawah (__).
- - Digunakan untuk mengimplementasikan polimorfisme dan mendefinisikan perilaku khusus pada kelas.
- - Contoh: `__add__`, `__str__`, `__len__`, dll.

Apa itu Metode Magic?

- Metode Magic:
- - Metode bawaan Python yang memungkinkan kustomisasi perilaku bawaan.
- - Membantu mengimplementasikan polimorfisme.
- Contoh Penggunaan Umum:
- 1. Operator overloading (+, -, *, dll).
- 2. Representasi string (`__str__` dan `__repr__`).
- 3. Dukungan fungsi bawaan (`len()`, `abs()`, dll).

Operator Overloading dengan __add__

- Contoh:
- class Vector:
- def __init__(self, x, y):
- self.x = x
- self.y = y
- def __add__(self, other):
- return Vector(self.x + other.x, self.y + other.y)

Mengubah Representasi String dengan `__str__` dan `__repr__`

- Contoh:
- `class Product:`
- `def __init__(self, name, price):`
- `self.name = name`
- `self.price = price`
- `def __str__(self):`
- `return f'{self.name}: ${self.price:.2f}'`

Mengatur Akses Atribut dengan __getattr__ dan __setattr__

- Contoh:
- class Person:
- def __init__(self, name, age):
- self.name = name
- self.age = age
- def __getattr__(self, item):
- return f"'{item}' not found"
- def __setattr__(self, key, value):
- if key == 'age' and value < 0:
- raise ValueError('Age cannot be negative')
- super().__setattr__(key, value)
- p = Person('Alice', 25)
- print(p.name) # Output: Alice
- print(p.gender) # Output: 'gender' not found
- p.age = -5 # Error: Age cannot be negative

Metode Magic Lainnya yang Sering Digunakan

- 1. `__len__`: Mendefinisikan perilaku fungsi `len()`.
- 2. `__eq__`, `__lt__`, dll: Mendefinisikan perbandingan objek (`==`, `<`, `>`, dll).
- 3. `__call__`: Membuat objek bisa dipanggil seperti fungsi.
- 4. `__getitem__`: Mendefinisikan akses elemen seperti list atau dictionary.
- Contoh `__len__`:
- `class Team:`
- `def __init__(self, members):`
- `self.members = members`
- `def __len__(self):`
- `return len(self.members)`
- `team = Team(['Alice', 'Bob', 'Charlie'])`
- `print(len(team))` # Output: 3

Keuntungan Menggunakan Metode Magic

- 1. Kustomisasi:
 - - Membuat kelas lebih fleksibel dan intuitif.
- 2. Efisiensi:
 - - Mengurangi duplikasi kode dengan memanfaatkan fungsi bawaan Python.
- 3. Integrasi Pythonic:
 - - Memberikan pengalaman penggunaan yang lebih sesuai dengan sintaks Python.

Kesimpulan

- - Metode Magic adalah alat yang kuat untuk mengimplementasikan polimorfisme dalam Python.
- - Memungkinkan kita untuk menyesuaikan perilaku kelas dan membuat kode lebih pythonic.
- - Dengan memanfaatkan metode magic, kita dapat menciptakan objek yang lebih fleksibel, modular, dan mudah digunakan.

Pertanyaan

- Apakah ada yang ingin ditanyakan mengenai polimorfisme dengan metode magic di Python?
- Terima kasih!